RBS Declarations

You can declare a class in RBS:

```
class ‹namespace::› classname module_types ‹< superclass superclass_types›
  body
end
```

An RBS class definition looks like a Ruby class definition except that both the class name and the superclass name can be augmented with type parameters. Those type parameters are used for *Generics*, and we'll talk about what those do and the syntax for them in RBS Generics, on page 490.

A class definition with the type modifiers looks like this:

```
class LinkedList[I] < List[I]
  # body
end
```

A module definition is similar, but a module has an additional type definition, like this:

```
module ‹namespace::› module name module_types ‹< superclass superclass_types›‹ : module-self-types›
  body
end
```

The module self type is type information about the kinds of classes that can include a module. So, you might have a whole class hierarchy underneath a class named AbstractPolicy and a module that you only want included within one of those classes:

```
module Adminish : AbstractPolicy
end
```

Where this gets a little more interesting is with the inclusion of an interface. RBS allows you to specify an interface that has a set of methods, and then any class that has those methods also matches the interface.

For example (we'll cover the method syntax in RBS Members, on page 487):

```
interface Postish
  def title: () -> String
  def description: () -> String
end

module Htmlable : Postish
end
```

In this case, the Htmlable module can only be included by classes that hit the Postish interface, meaning they have methods called title and description.

An interface declaration is basically the same as a module:

```
interface ‹namespace::› module name module_types ‹< superclass superclass_types›
  body
end
```

You can also declare aliases using the keyword type, which allows you to specify shortcuts:

```
type alias type-parameters = type
```